

DESIGNING DNNS

Efficiency and scale

Adrian Jackson

a.jackson@epcc.ed.ac.uk



THE UNIVERSITY
of EDINBURGH



ML performance

- Performance of ML is dependent on resource usage and resource requirements
 - High computational requirements require more resources or longer time
 - Computational requirements depend on network design and amount of data
 - Also depend on input and output sizes
- Designing/choosing low(est) resource network architecture a big part of achieving good performance/scaling for ML
 - Reducing computational complexity
 - Reducing communication requirements
 - Reducing convergences time

Training

- **What is DNN Training?**

- Update tunable parameters (weights and biases) in the network to match seen data

- **Why is it Important?**

- Allows learning patterns to reproduce data from inputs
- Enables (or tries to enable) recognising general patterns so new data can be processed

- **Key Components**

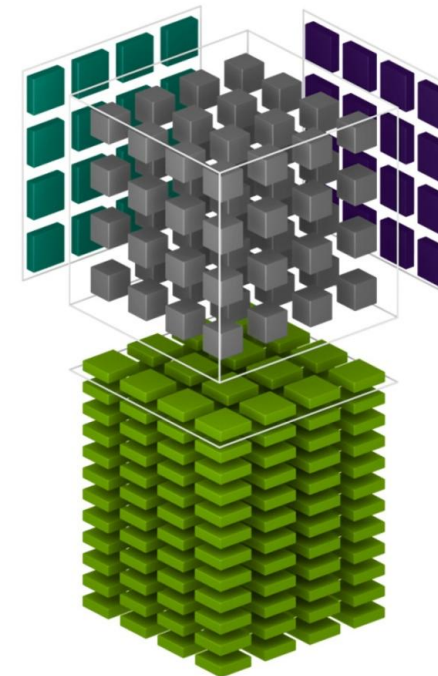
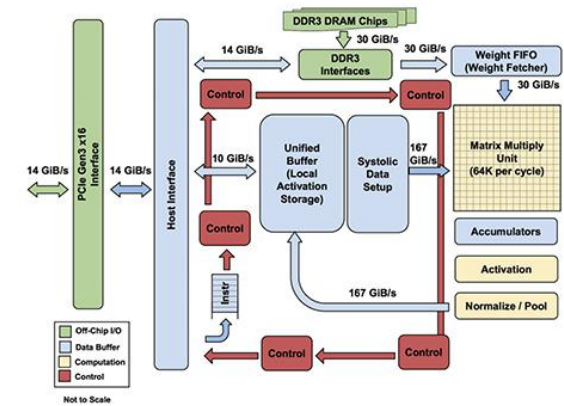
- **Data:** High-quality and diverse datasets are essential for training DNNs
- **Model Architecture:** Designing the neural network architecture, including the number of layers, neurons, and activation functions
- **Loss Function:** Selecting an appropriate loss function to measure the difference between predicted and actual values
- **Optimisation:** Using optimisation algorithms like gradient descent to update model parameters
- **Hyperparameters:** Tuning hyperparameters such as learning rate, batch size, and regularisation

- **Challenges in DNN Training:**

- **Overfitting:** Only fit the training data, not general functionality
- **Vanishing and Exploding Gradients:** Stops the training converging to sensible results
- **Computational Resources:** Training large DNNs often requires significant computational power and time, grows with network or dataset size

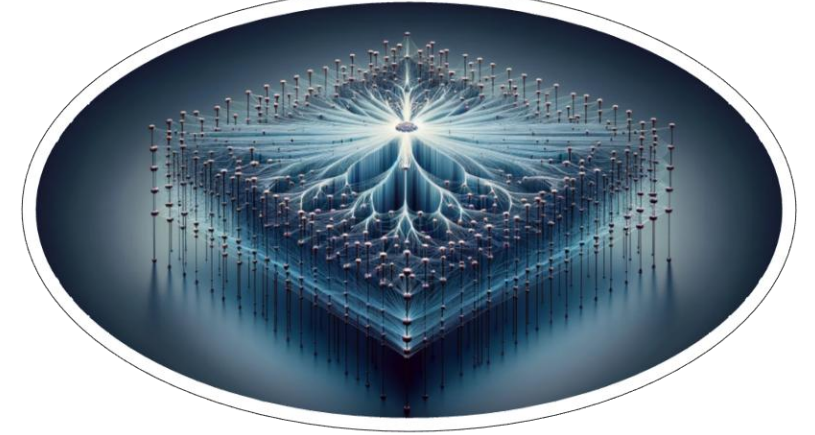
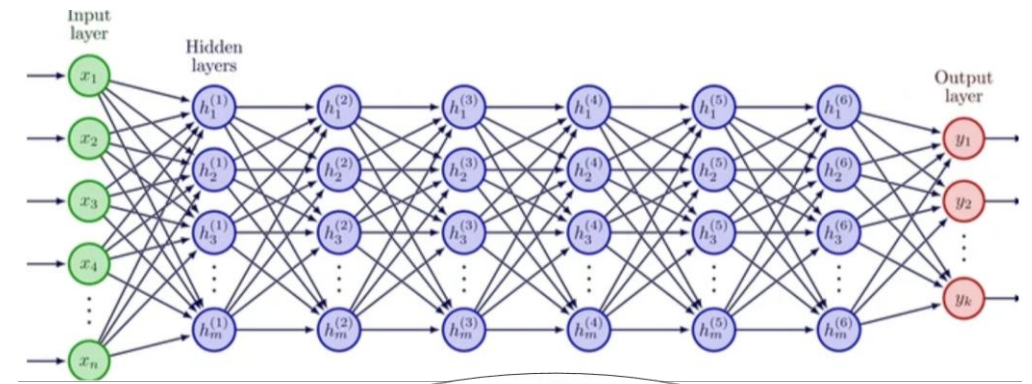
HPC hardware

- Looking for compute to be the bottleneck if possible
 - Ideally not memory bandwidth, compute or storage bound
 - Approach has been to maximise hardware to improve performance
- Compute
 - Using accelerators or specialised processing to reduce architectural overheads
- Memory
 - Highest bandwidth or capacity memory
 - HBM or GDDR5
- Network
 - Low latency high bandwidth connections
 - Reduce communication capacities
- Storage
 - Training data and weight checkpoints



Model architecture

- Choosing a DNN structure depends on:
 - Task to be undertaken
 - Data to be processed/produced
 - Amount/quality of training data available
 - Relationship between data points
- Choices that can be made
 - Architectural features: Connections, components, and common layers
 - Sizes: Depth and width of layers
 - Hyperparameters: Tunable parameters that can affect performance or functionality



Efficiency

- Different aspects to consider when looking to optimise the efficiency of DNN structures
 - Computational Efficiency
 - Memory Efficiency
- Need to consider both training and inference
 - Training dominates cost, but where will inference happen?
- Energy Efficiency
 - Computational efficiency is often correlated with energy efficiency, but not always
- Data Efficiency
 - Training when not much data is available
 - Transfer learning possible here
- Computational efficiency
 - Parameter Efficiency: network pruning, fewer parameters, different structures

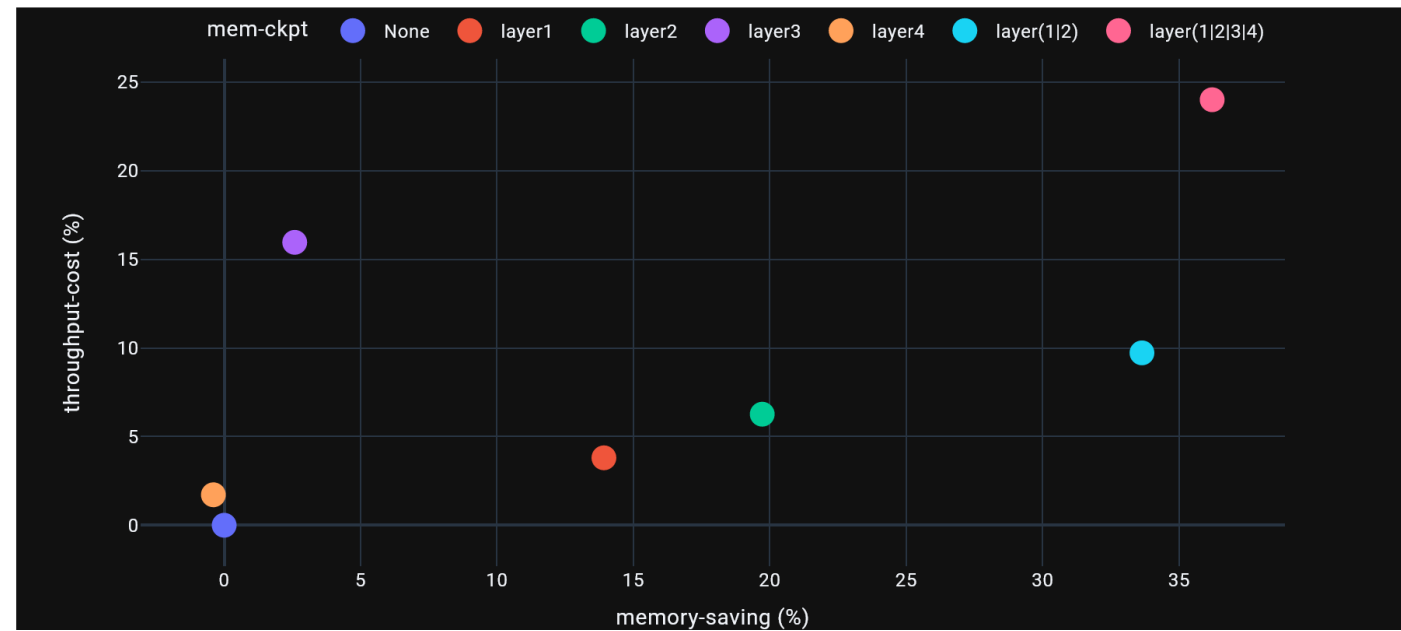
Efficiency

- Memory efficiency

- Similar to compute
- Precision (Quantisation)
- Pooling
- Checking pointing in back propagation

- Generalisation:

- Ensuring DNN will work on the range of inputs required
- Reduces need to re-train and re-specialise networks
- Trade off between size and scope
- Addressing overfitting and underfitting



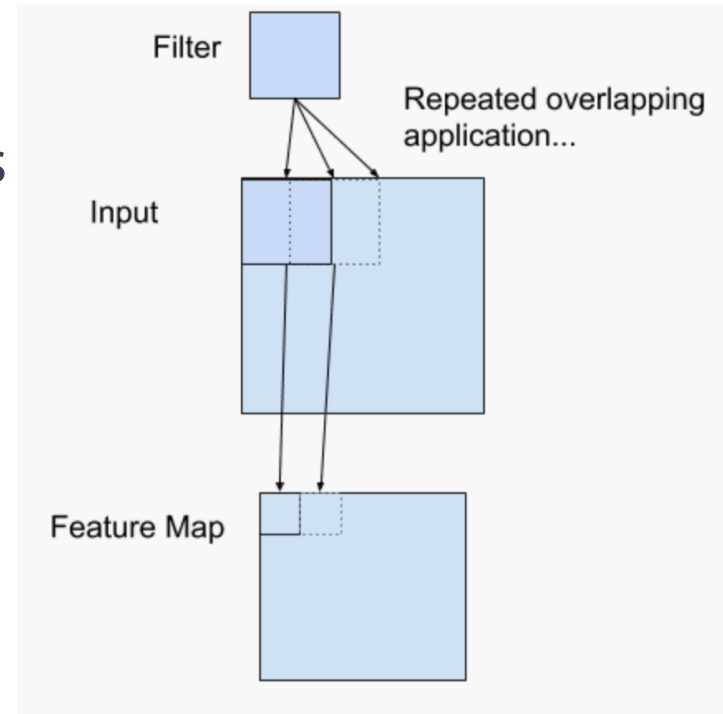
Data preparation and processing

- Number of approaches can speedup training convergence
 - Normalisation to fixed range or mean/std (i.e. 0 to 1, 0 mean with 1 std) reduces outlier impact on training
 - Cleaning can remove redundant or overly noisy data (reduce epoch size)
 - Expansion/Augmentation can add data to improve generalizability
 - Encoding/embedding/one-hot encoding

City		Houston	Rome	Madrid	London
Houston	one-hot encoding →	1	0	0	0
Rome		0	1	0	0
Madrid		0	0	1	0
London		0	0	0	1

Simplifying architectures

- Architecture choices
 - Start with as simple an architecture as possible (minimum depth and width)
 - Choose cheaper activation functions
 - Choose layers that are cheaper if possible, i.e. convolutions, pooling
 - Reduce weight space and create parallel operations
 - Prune connections and contributions
 - Reduce datatypes
 - Gradient regularisation



Activation functions

- ReLU (Rectified Linear Unit)

$$\max(0, z)$$

- Leaky ReLU

$$\max(\alpha z, z)$$

- Sigmoid (used for probability output)

$$\frac{1}{1 + e^{-z}}$$

- Tanh (Hyperbolic Tangent)

$$\frac{2}{1 + e^{-2z}} - 1$$

- Softmax (used for multi-class classification in output layers)

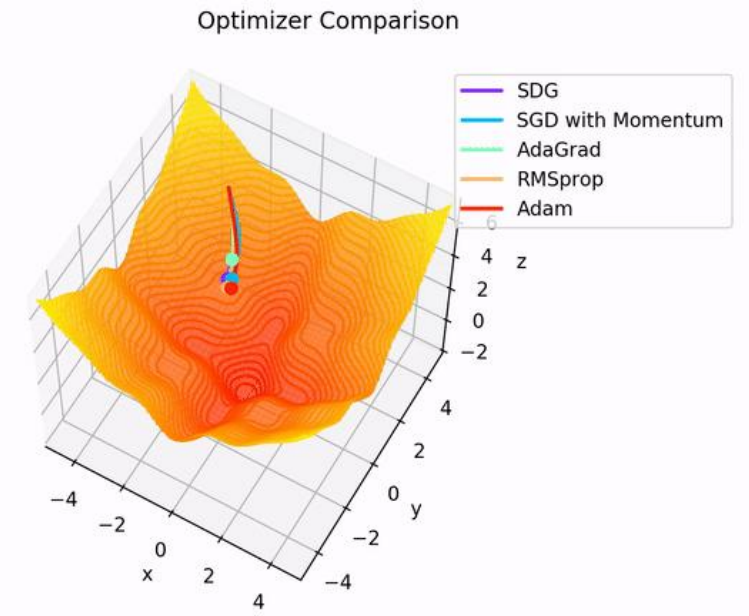
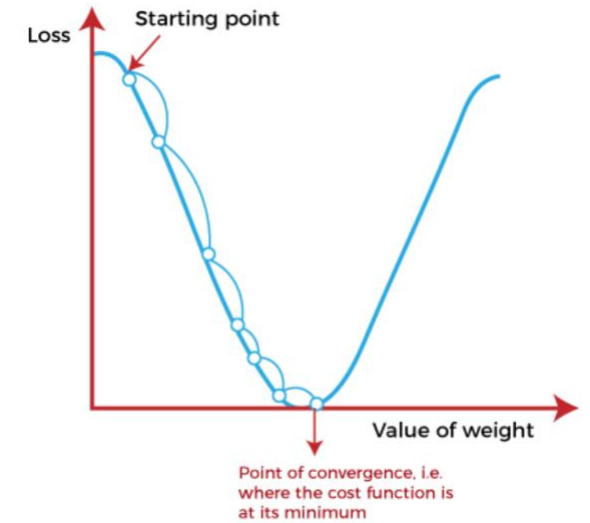
$$\frac{e^{z_i}}{\sum_{j=1}^k e^{z_j}}$$

Regularisation

- Ensure networks don't overfit training data
- Drop-out
 - Remove contributions of neurons selectively as training
 - Ensures network not reliant on specific parts for output
 - Generalises and introduces resilience
- Loss-function optimisations
 - L1 (Lasso regularisation)
 - Extra loss function term, aims at reducing the number/size of weights
 - Effectively reduces model features
 - Absolute value of the weights multiplied by lambda (regularisation parameter)
 - L2 (Ridge regularisation)
 - Extra loss function contribution, aims at reducing weight sizes, penalizes large weights
 - Reduces overfitting
 - Add square of the size of the weights
- Stopping criteria
 - Deciding when to finish training the network
 - Balance between precision and cost

Optimiser optimisations

- Fixed learning rate
 - Sets the step size of updates
 - Delta on the gradient value
- Learning rate schedulers
 - Variable learning rates through training process
 - Like adaptive time stepping
 - Exponential scheduling, piecewise constant scheduling, etc...
- Momentum
 - Additional value that includes the previous gradients in gradient update (velocity)
 - Moves faster through search space/avoids local minima
 - Requires tuning (too big or too small can be bad)
- Move to more complex optimisers
 - ADAM: Adaptive Moment Estimation
 - Noisy or sparse gradient landscapes
 - Non-convex spaces, etc...

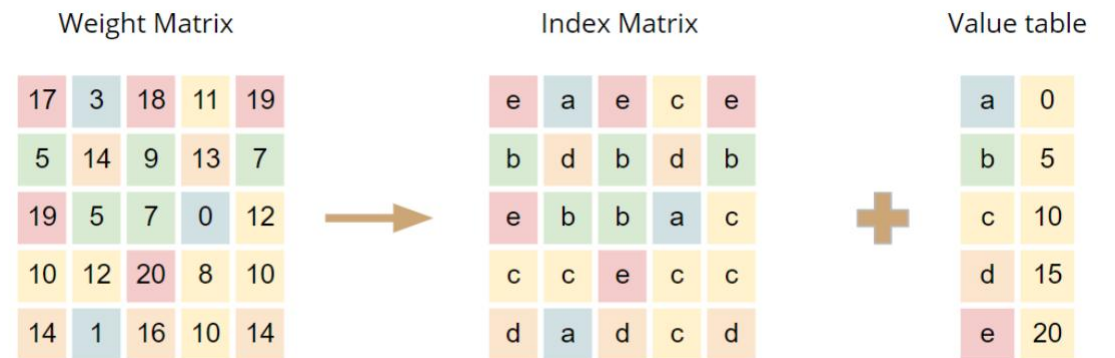


Quantisation

- Utilising optimised data types to improve memory or compute performance
 - Leverage the over-parametisation of DNN structures
 - Network parameters and activation function data types
 - 32-bit floats to 16-bit floats, 8-bit ints, etc...
 - Reduces memory footprint, communication costs
 - Can be faster on hardware with data type support
 - 8-bit int support on accelerators
- Training and inference optimisation choices
 - Quantisation aware training
 - Post training quantisation
- Post training
 - Convert weights to lower precision
- Can reduce accuracy
 - Lose ability to differentiate
 - Require higher precision accumulators etc...

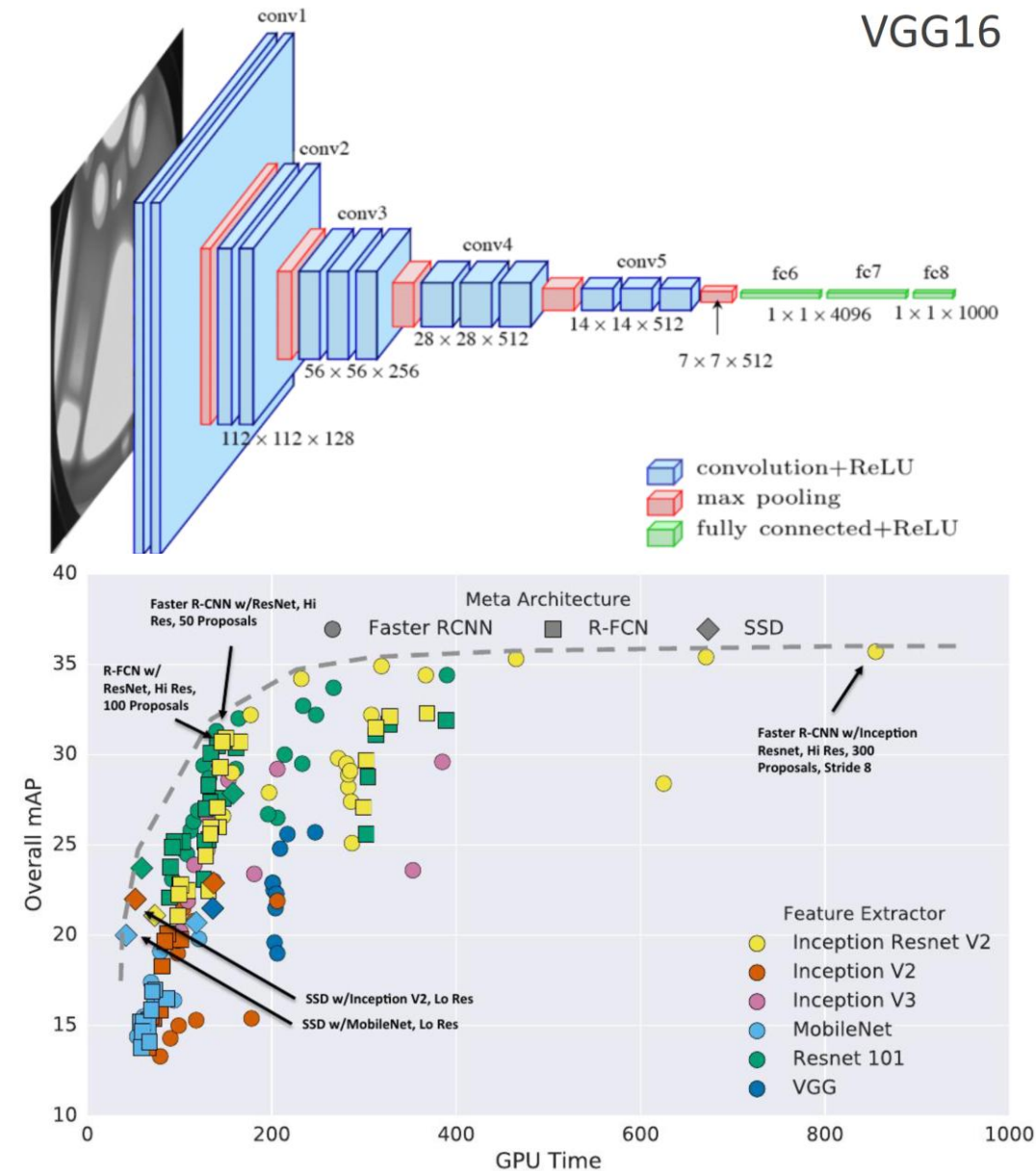
Weight sharing

- Compress weights into look-up tables
 - Doesn't reduce number of weights
 - But reduces storage required
 - Requires reduced weight range (coupled with quantisation) or has function
- CNN and RNN do weight sharing as part of their design
 - Filter kernels on CNN
 - Helps ignore positions etc...



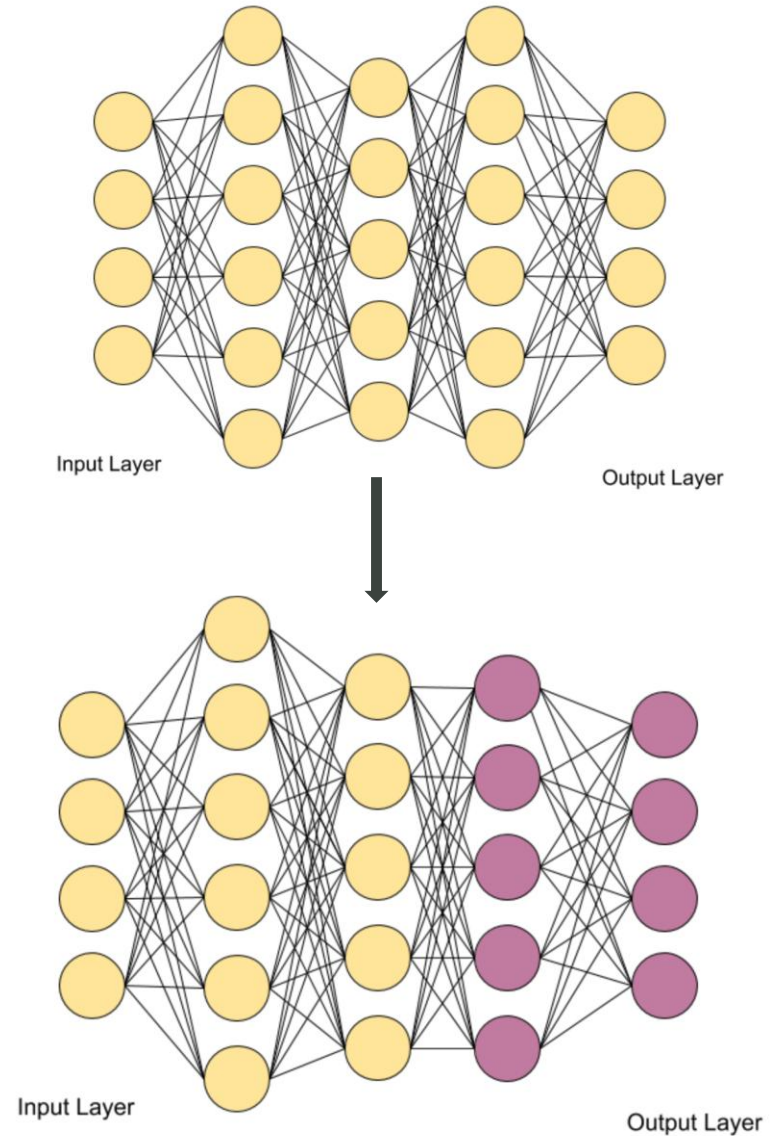
Transfer learning

- Training is expensive
 - From scratch requires many iterations
 - Costly even if can parallelise
- Training complex models requires a lot of data
 - Millions of inputs to tune hundreds of millions of parameters
- Transfer learning can reduce/mitigate these issues
 - Take existing network trained on large dataset
 - Retrain for new task or on new dataset



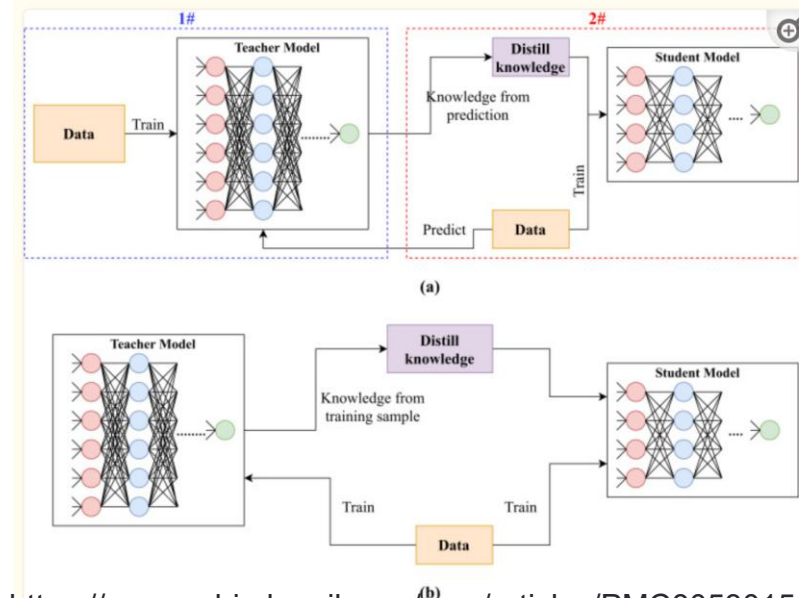
Transfer learning

- Take existing weights/architecture and specialise
 - Don't change most of the system
 - Optimise a small part
- Change the classifier
 - Update the fully connected layer
 - Train with the relevant datasets available
- Change feature layers
 - Freeze most of the weights
 - Only update weights in later layers
- Increase layer capacity
 - Freeze existing weights
 - Add extra depth or channels
- Retrain all layers
 - Requires similar domains
 - Requires good data to match



Transfer learning – Inference Optimisation

- Knowledge distillation
 - Inference doesn't need the complexity of training
 - Inference can be very resource constrained
- Train two networks
 - Teacher network: Full complexity, high precision
 - Student network: Simpler network
- Modify student network loss function
 - Optimise on Teacher network hidden layers as well as output loss
 - Can use different parts of the networks for those loss function comparison

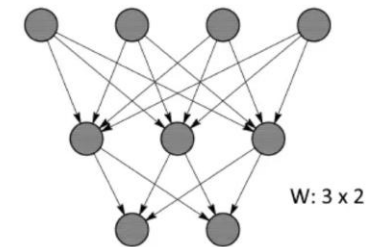


<https://www.ncbi.nlm.nih.gov/pmc/articles/PMC8053015/>

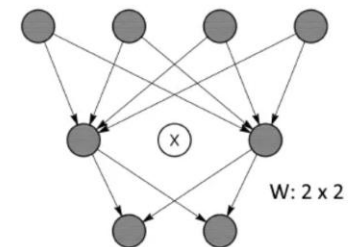
Reference	Targeted architecture	Utilized data	Reduction in accuracy compared to teacher	Improvement in accuracy compared to baseline	Reduction in size
Offline distillation					
Lopes, Fenu & Starner (2017)	CNN	MNIST	4.8%	5.699% (decrease)	50%
Yim et al. (2017)	ResNet	CIFAR-10	0.3043% (increase)	-	-
Gao et al. (2018)	ResNet	CIFAR-100	2.889%	7.813%	96.20%
Wang et al. (2019)	U-Net	Janelia (Peng et al., 2015)	-	-	78.99%
He et al. (2019)	MobileNetV2	PASCAL (Everingham et al., 2010)	4.868% (mIOU)	-	92.13%
Heo et al. (2019)	WRN	ImageNet to MIT scene (Quattoni & Torralba, 2009)	6.191% (increase)	14.123%	70.66%
Li et al. (2019)	CNN	UIUC-Sports (Li et al., 2010)	7.431%	16.89%	95.86%
Liu et al. (2019)	ResNet	CIFAR10	0.831%	2.637%	73.59%
Online distillation					
Zhou et al. (2018)	WRN	CIFAR-10	1.006%	1.37%	66%

Inference optimisation

- Network pruning
 - Removing neurons with small or zero contributions
 - Reduce network size (memory) or computational requirements
 - Can be iterative (pruning, tuning, pruning, etc...)
- Structured pruning
 - Connections to neuron removed
 - Reduces weight matrix
- Magnitude pruning
 - Setting small weights to zero
 - Same side weights matrix and connections
 - Sparsification of the network
 - Can be done automatically with loss function optimisation



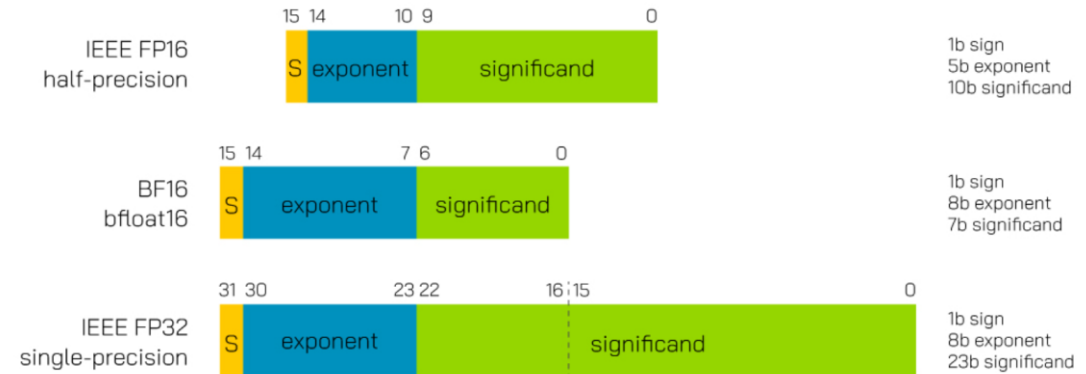
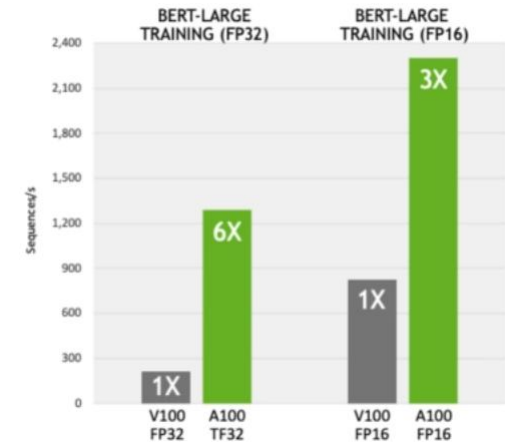
Before pruning



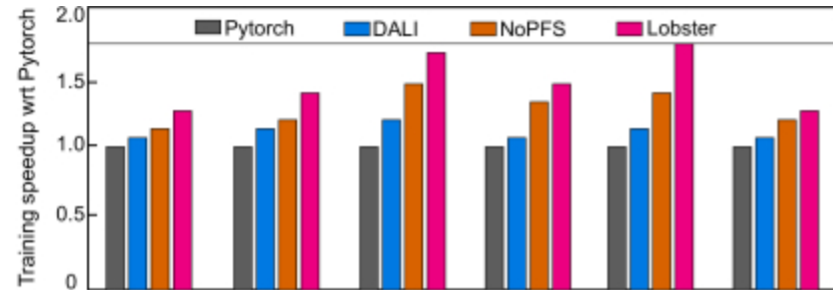
After pruning

Efficient use of hardware

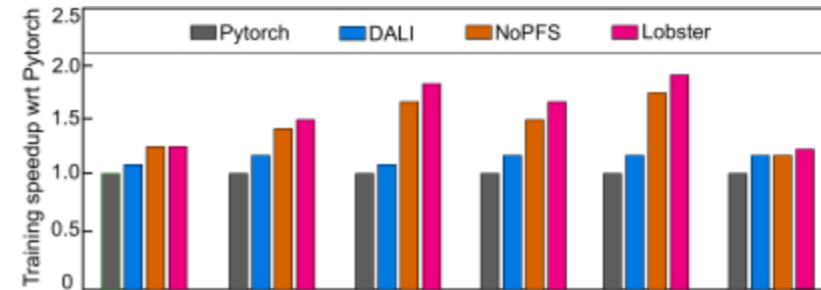
- Targeting all the previously discussed optimisation for the hardware being used
 - Data types supported on accelerator hardware
 - FP64, FP32, FP16, BFLOAT16, TF32
- Kernel tuning on hardware
 - Matching kernels to GPU parameters
- Overlap communications
- Overlap data transfers
- Parallelisation



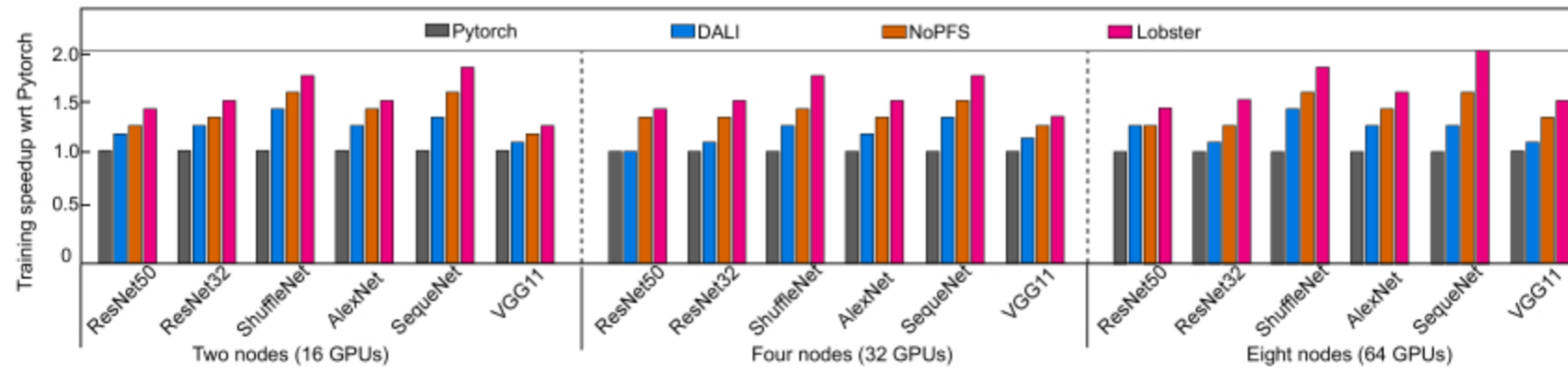
Examples



(a) Multi-GPU training in a single node with ImageNet-1K



(b) Multi-GPU training in a single node with ImageNet-22K



(c) Distributed training across nodes with ImageNet-22K

<https://dl.acm.org/doi/fullHtml/10.1145/3545008.3545090>

Summary

- Aiming for optimal performance of DNNs
 - Computationally efficient: Minimum required active components
 - Memory efficient: Smallest required active components
 - Generalisable: Match domain without requiring retraining
- Like performance optimisation...
 - ...need to make sure model is correct and functional first
 - Satisfy computational requirements first, then optimise
 - But, choosing a good starting point (architecture) can really help
 - And understanding your requirements and tolerances is key
- Requirements for future inference are also important